SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Cyber Security

Practical no: 2

Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CODE:-

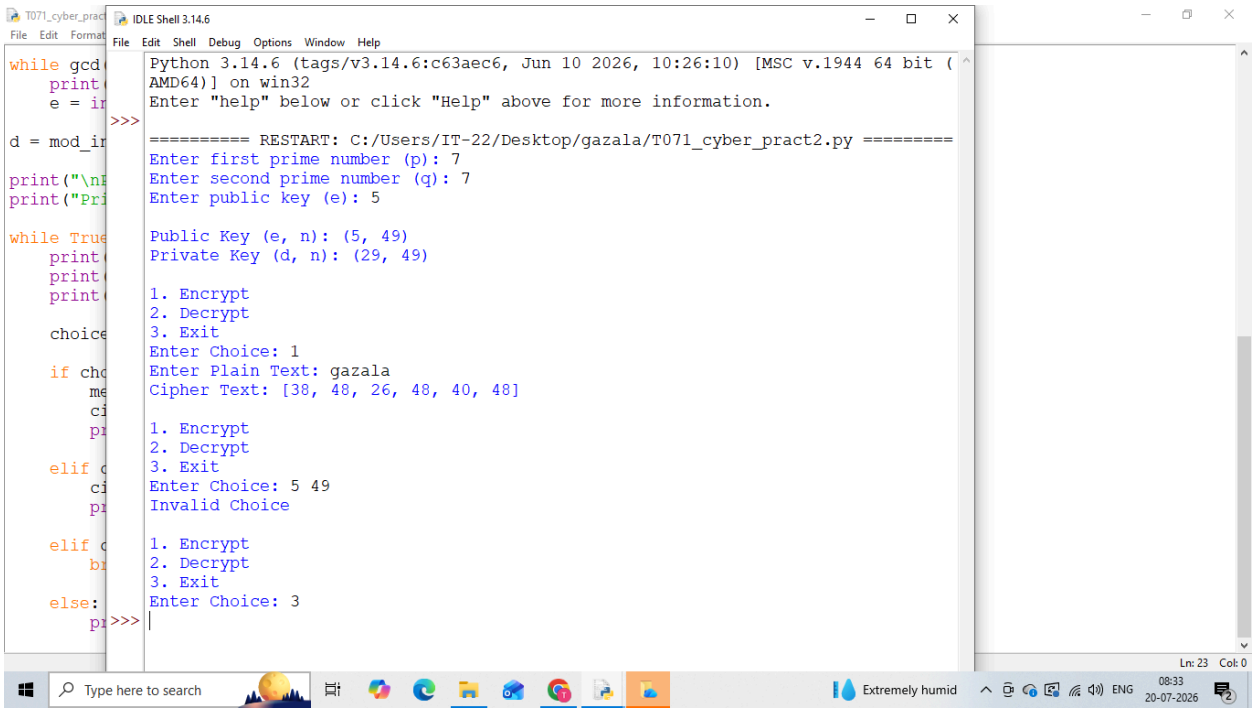
```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (e * d) % phi == 1:
            return d
    return None

def encrypt(message, e, n):
    cipher = []
    for ch in message:
        cipher.append(pow(ord(ch), e, n))
    return cipher

def decrypt(cipher, d, n):
    message = ""
    for num in cipher:
        message += chr(pow(num, d, n))
    return message

p = int(input("Enter first prime number (p): "))
q = int(input("Enter second prime number (q): "))

n = p * q
phi = (p - 1) * (q - 1)

e = int(input("Enter public key (e): "))

while gcd(e, phi) != 1:
    print("e must be coprime with phi.")
    e = int(input("Enter another value for e: "))
```

```
d = mod_inverse(e, phi)

print("\nPublic Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))

while True:
    print("\n1. Encrypt")
    print("2. Decrypt")
    print("3. Exit")

    choice = input("Enter Choice: ")

    if choice == '1':
        message = input("Enter Plain Text: ")
        cipher = encrypt(message, e, n)
        print("Cipher Text:", cipher)

    elif choice == '2':
        cipher = list(map(int, input("Enter Cipher Numbers (space separated): ").split()))
        print("Plain Text:", decrypt(cipher, d, n))

    elif choice == '3':
        break

    else:
        print("Invalid Choice")
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Cyber Security

Output:-



GUI:-

```
from tkinter import *
from tkinter import messagebox

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (e * d) % phi == 1:
            return d
    return None

def generate_keys():
    try:
        global e, d, n

        p = int(p_entry.get())
        q = int(q_entry.get())
        e = int(e_entry.get())
```

```
        phi = (p - 1) * (q - 1)
        n = p * q

        if gcd(e, phi) != 1:
            messagebox.showerror("Error", "e must be coprime with φ(n).")
            return

        d = mod_inverse(e, phi)

        if d is None:
            messagebox.showerror("Error", "Invalid value of e.")
            return

        public_label.config(text=f"Public Key : ({e}, {n})")
        private_label.config(text=f"Private Key : ({d}, {n})")

    except:
        messagebox.showerror("Error", "Enter valid numeric values.")

def encrypt():
    try:
        msg = message_entry.get()
        cipher = [pow(ord(ch), e, n) for ch in msg]
        result.delete(1.0, END)
        result.insert(END, " ".join(map(str, cipher)))
    except:
        messagebox.showerror("Error", "Generate keys first.")

def decrypt():
    try:
        cipher = list(map(int, message_entry.get().split()))
        plain = "".join(chr(pow(num, d, n)) for num in cipher)
        result.delete(1.0, END)
        result.insert(END, plain)
    except:
        messagebox.showerror("Error", "Enter valid cipher numbers.")

def clear():
    p_entry.delete(0, END)
    q_entry.delete(0, END)
```

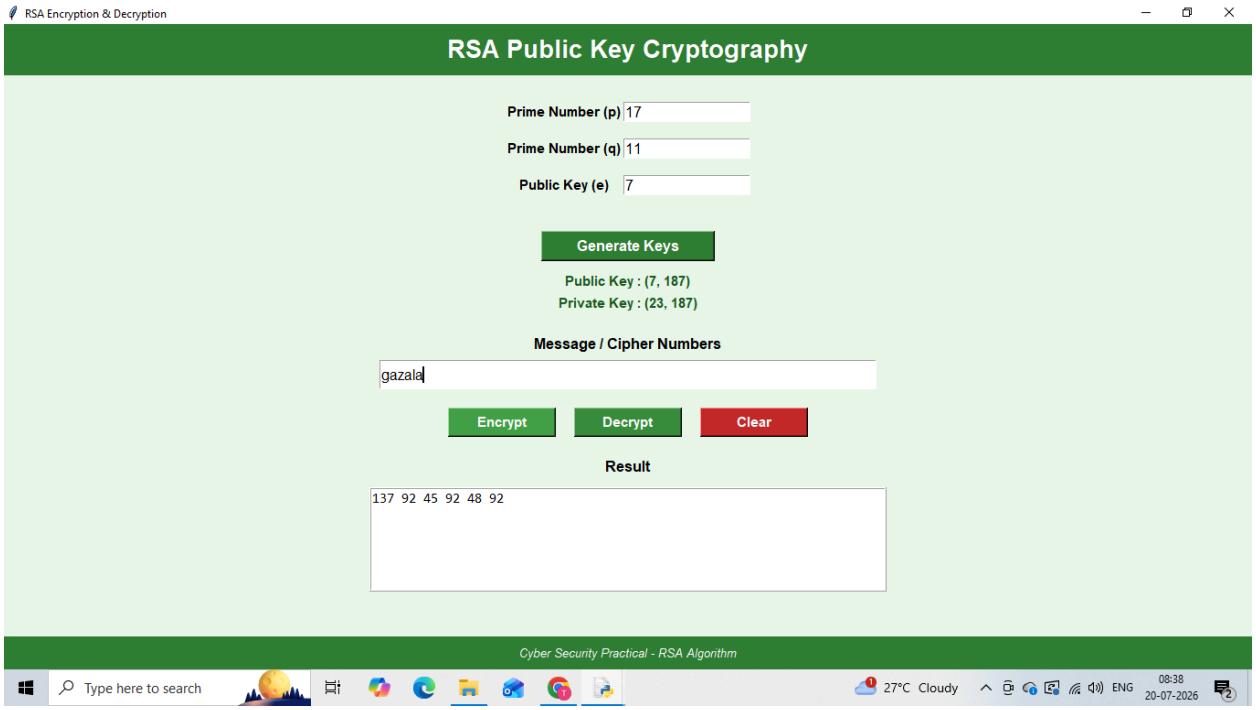
```
    e_entry.delete(0, END)
    message_entry.delete(0, END)
    result.delete(1.0, END)
    public_label.config(text="Public Key :")
    private_label.config(text="Private Key :")

root = Tk()
root.title("RSA Encryption & Decryption")
root.geometry("700x600")
root.configure(bg="#E8F5E9")

title = Label(root,
              text="RSA Public Key Cryptography",
              font=("Helvetica", 20, "bold"),
              bg="#2E7D32",
              fg="white",
              pady=10)
title.pack(fill=X)

frame = Frame(root, bg="#E8F5E9")
frame.pack(pady=20)

Label(frame, text="Prime Number (p)", bg="#E8F5E9",
      font=("Arial", 11, "bold")).grid(row=0, column=0, pady=8)

p_entry = Entry(frame, font=("Arial", 12), width=15)
p_entry.grid(row=0, column=1)

Label(frame, text="Prime Number (q)", bg="#E8F5E9",
      font=("Arial", 11, "bold")).grid(row=1, column=0, pady=8)

q_entry = Entry(frame, font=("Arial", 12), width=15)
q_entry.grid(row=1, column=1)

Label(frame, text="Public Key (e)", bg="#E8F5E9",
      font=("Arial", 11, "bold")).grid(row=2, column=0, pady=8)

e_entry = Entry(frame, font=("Arial", 12), width=15)
e_entry.grid(row=2, column=1)

Button(root,
```

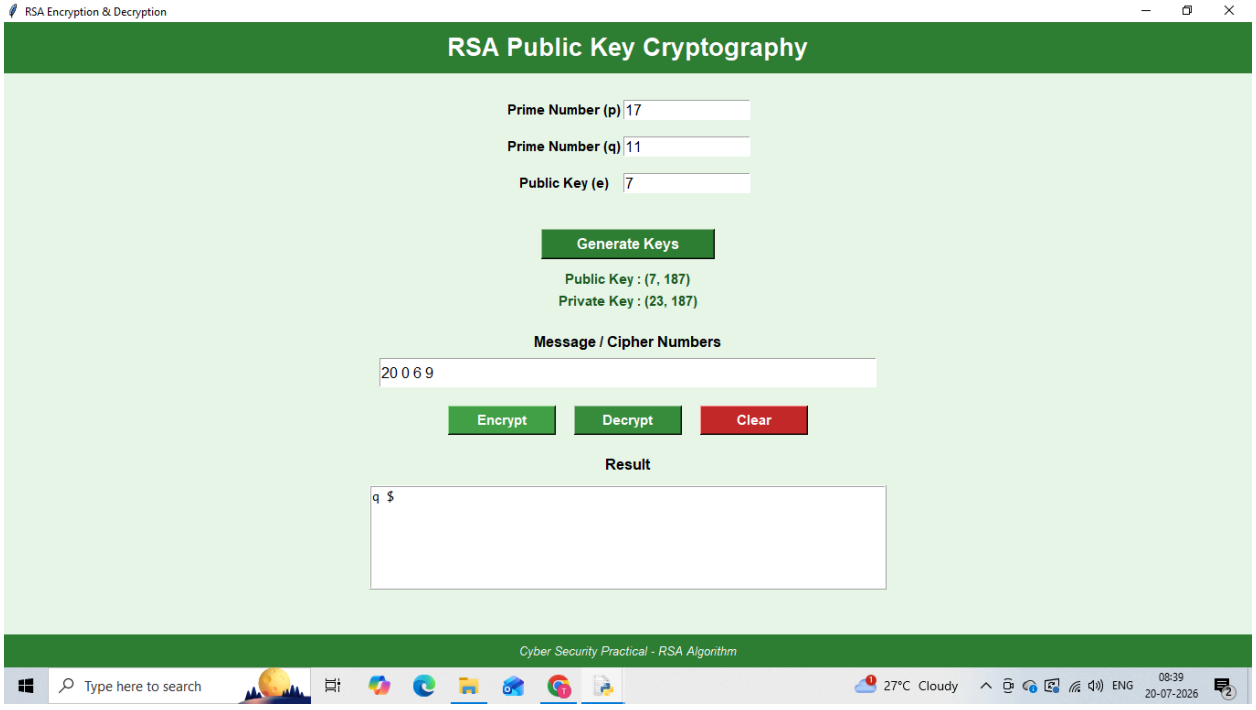
```
        text="Generate Keys",
        bg="#2E7D32",
        fg="white",
        font=("Arial", 12, "bold"),
        width=18,
        command=generate_keys).pack(pady=10)

public_label = Label(root,
                     text="Public Key :",
                     font=("Arial", 11, "bold"),
                     bg="#E8F5E9",
                     fg="#1B5E20")
public_label.pack()

private_label = Label(root,
                      text="Private Key :",
                      font=("Arial", 11, "bold"),
                      bg="#E8F5E9",
                      fg="#1B5E20")
private_label.pack()

Label(root,
      text="Message / Cipher Numbers",
      font=("Arial", 12, "bold"),
      bg="#E8F5E9").pack(pady=(20,5))

message_entry = Entry(root, font=("Arial", 12), width=60)
message_entry.pack(ipady=5)

button_frame = Frame(root, bg="#E8F5E9")
button_frame.pack(pady=20)

Button(button_frame,
        text="Encrypt",
        bg="#43A047",
        fg="white",
        font=("Arial", 11, "bold"),
        width=12,
        command=encrypt).grid(row=0, column=0, padx=10)

Button(button_frame,
```

```
        text="Decrypt",
        bg="#388E3C",
        fg="white",
        font=("Arial", 11, "bold"),
        width=12,
        command=decrypt).grid(row=0, column=1, padx=10)

Button(button_frame,
        text="Clear",
        bg="#C62828",
        fg="white",
        font=("Arial", 11, "bold"),
        width=12,
        command=clear).grid(row=0, column=2, padx=10)

Label(root,
      text="Result",
      font=("Arial", 12, "bold"),
      bg="#E8F5E9").pack()

result = Text(root,
              height=6,
              width=70,
              font=("Consolas", 11),
              bd=2,
              relief=RIDGE)
result.pack(pady=10)

footer = Label(root,
               text="Cyber Security Practical - RSA Algorithm",
               font=("Arial", 10, "italic"),
               bg="#2E7D32",
               fg="white",
               pady=8)
footer.pack(fill=X, side=BOTTOM)

root.mainloop()
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Cyber Security





Alam Gazala T071

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Cyber Security

Alam Gazala T071